

Módulo: Programación Multimedia y dispositivos móviles.
Unidad de Trabajo N°1: Introducción al desarrollo de aplicaciones móviles.
Actividad de Clase 4: Controles RadioGroup, RadioButton, CheckBox y Spinner y ViewBinding.

1. Objetivo general.

Manejar diferentes componentes (RadioGroup, RadioButton, Checkbox y Spinner) en Android Studio.

2. Metodología.

El alumnado realizará la app de forma individual.

3. Descripción.

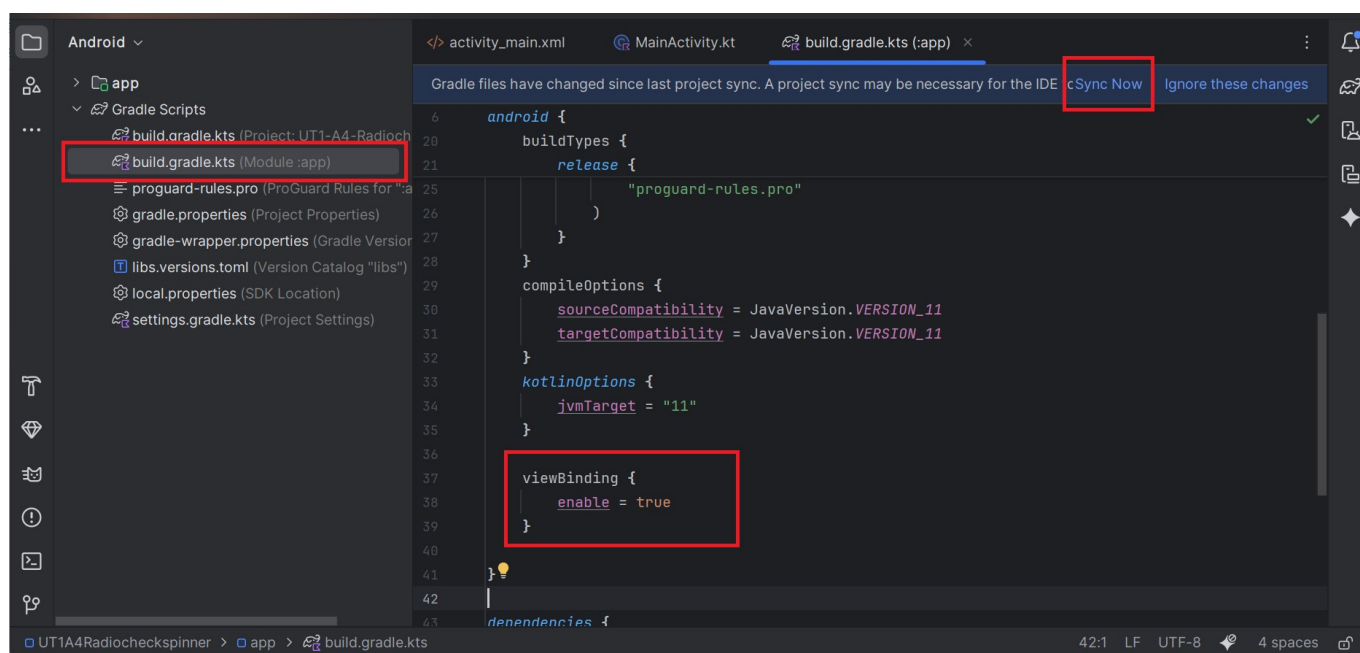
Realiza lo siguiente:

Primera Parte: Utilizar View Binding en vez findViewById

1. Para vincular las vistas (los componentes) hemos utilizado hasta ahora findViewById . Como en el siguiente ejemplo donde accedemos a un componente definido, en este caso un spinner que utilizaremos más adelante en la actividad.

```
//Accedemos al spinner  
val spin = findViewById<Spinner>(R.id.spinner2)
```

2. Como hemos visto este código funciona. Pero tendremos que hacer lo mismo con cada componente que queramos usar generando un montón de código repetido. Por ello, tenemos otra forma de hacerlo y es usando **view binding**. Para configurarlo debemos ir a **build.gradle.kts (Module :app)** y allí buscaremos la sección **android {}** y en esta sección añadiremos lo siguiente:



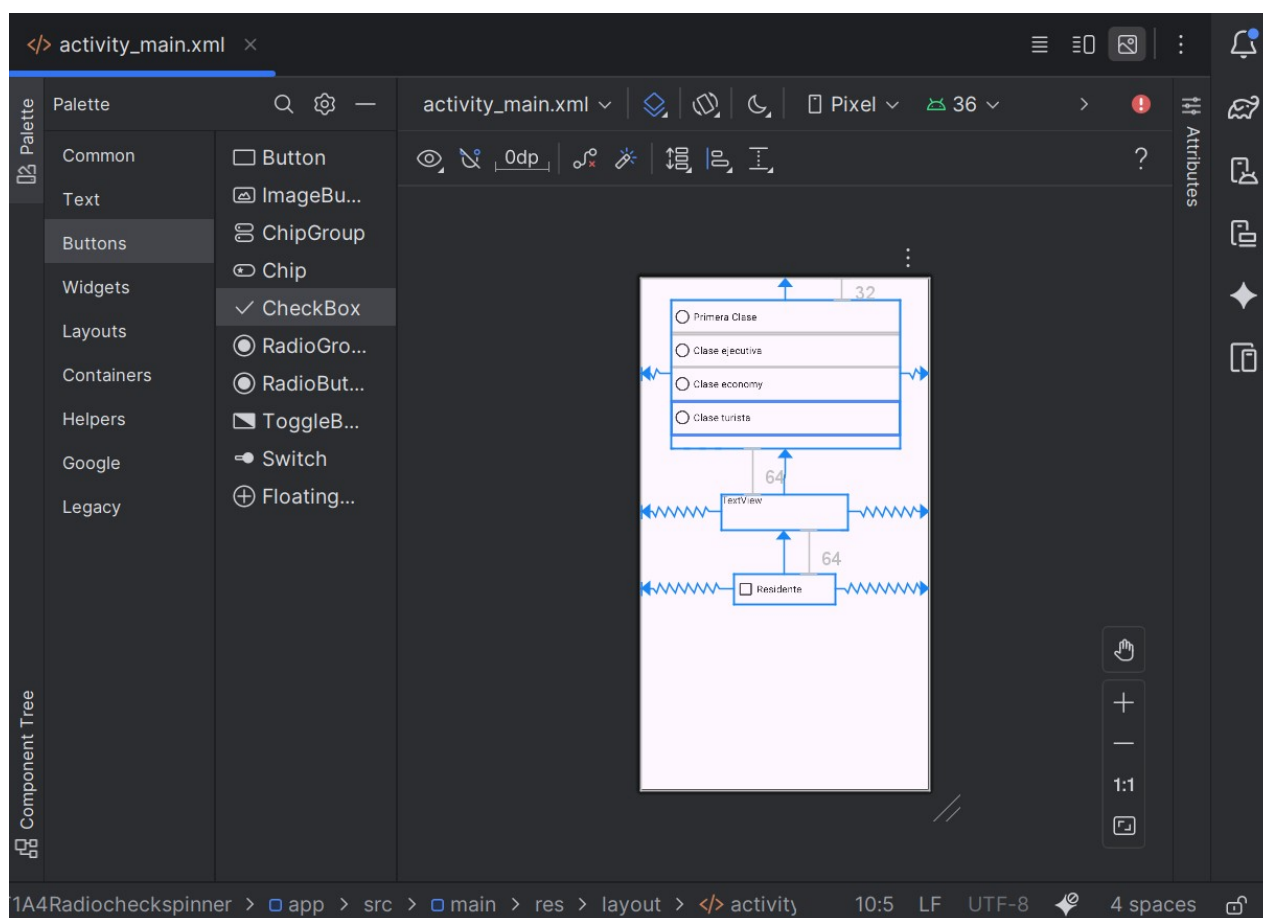
3. Sincronizada el Gradle.

Segunda Parte: Controles RadioGroup, RadioButton y CheckBox.

4. Vamos a crear una app que va a elegir los asientos de un avión. Los tipos de asientos son las siguientes:
 - Primera Clase, Clase ejecutiva o business, Clase premium economy, Clase turista o económica.
 - El usuario sólo podrá elegir un tipo de asiento.
 - Información al usuario que sólo puede seleccionar una zona.
 - Marcar si el usuario es residente o no.
5. Puedes añadir los controles de forma gráfica, arrastrando los Radiobutton dentro del Radiogroup, o bien, en modo texto en el XML (prueba a hacerlo de las dos maneras).
6. La **Activity principal** contendrá los siguientes controles:
7. Un **Radiogroup** y dentro un **Radiobutton** para cada tipo de asiento.
8. Un **TextView** para mostrar información al usuario que sólo puede seleccionar el tipo de asiento
9. Un **Checkbox** para que el usuario seleccione si es o no residente.

Nota: Las imágenes de referencia que aparecen en el documento no son las solicitadas en el programa solicitado.

10. La Activity quedaría de forma similar a la que aparece en la siguiente imagen donde vemos el RadioGroup, los RadioButton, TextView y Checkbox en modo diseño: En las imágenes tiene los nombres por defectos (realizado antes de modificar las propiedades **id** y **text**)



11. Una vez finalizado el diseño de la Activity donde hemos trabajado con el archivo xml (**activity_main.xml**) donde se definen los controles visuales de la ventana que estamos creando. Abrimos el archivo **MainActivity.kt**.

12. A continuación, vamos a ver el código de la clase **MainActivity** pero viendo las diferentes partes. Vamos a empezar primero por el manejo del **View Binding**. una vez

activado el view binding, cada activity que creamos, creará por detrás el binding, es decir, si nosotros tenemos **MainActivity** que tiene un layout llamado activity_main.xml, automáticamente se creará **ActivityMainBinding**. Lo primero es crear una variable de clase que se del tipo **ActivityMainBinding** pero como dicha variable la vamos a **inicializar** más tarde (en el método **onCreate()**) la vamos a declarar del tipo **lateinit** este tipo en **Kotlin** nos permite declararla e inicializarla posteriormente.

```
class MainActivity : AppCompatActivity() {  
  
    // Declaramos la variable del tipo ActivityMainBinding. La podemos inicializar más tarde porque es lateinit  
    private lateinit var binding: ActivityMainBinding
```

13. Ahora inicializaremos la variable, en el método **onCreate()** de nuestra activity. Después solo tendremos que cambiar la función **setContentView** (que ahora mismo mandamos el layout) y le ponemos la ruta de nuestro binding.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    enableEdgeToEdge()  
    // Inicializamos la variable binding  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    // Mostramos la vista raíz  
    setContentView(binding.root)
```

14. Ahora ya podríamos acceder a nuestros componentes simplemente usando **binding.LaIdDelComponente**. Como podemos observar a continuación donde accedemos a los componentes sin tener que utilizar el **findViewById**.

15. Para poder acceder a los objetos visuales definidos en el archivo activity_main.xml (El radioGroup, el TextView y el checkbox). .

```
ViewCompat.setOnApplyWindowInsetsListener( view = findViewById( id = R.id.main)) { v, in  
    val systemBars = insets.getInsets( typeMask = WindowInsetsCompat.Type.systemBars())  
    v.setPadding( left = systemBars.left, top = systemBars.top, right = systemBars.right,  
insets  
    }  
  
    // Accedemos a los componentes definidos en la vista (archivo xml)  
    val rgasientos = binding.rgasientos  
    val tvmensaje = binding.tvmensaje  
    val cbresidente = binding.cbresidente
```

16. Ahora vemos como creamos el listener para el RadioGroup. En **Kotlin** en vez de utilizar **switch case**. Utilizamos **when** que es similar.

```
rgasientos.setOnCheckedChangeListener { p0, p1 ->
    when(p1) {
        binding.rbprimera.id -> {
            tvmensaje.text = binding.rbprimera.text
        }

        binding.rbejecutiva.id -> {
            tvmensaje.text = binding.rbejecutiva.text
        }

        binding.rbeconomy.id -> {
            tvmensaje.text = binding.rbeconomy.text
        }

        binding.rbturista.id -> {
            tvmensaje.text = binding.rbturista.text
        }
    }
}
```

17. Y ahora vemos el listener para el CheckBox

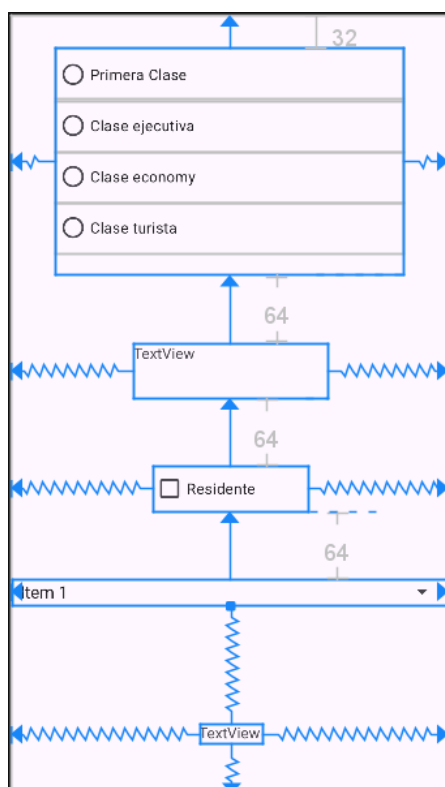
```
cbresidente.setOnCheckedChangeListener { p0, p1 ->
    // p0 representa el boton
    // p1 representa si esta seleccionado o no
    var mensaje = ""
    if (p1) {
        mensaje = cbresidente.text.toString()
    } else {
        mensaje = "No ${cbresidente.text.toString()}"
    }
    Toast.makeText(context = this, text = mensaje, duration = Toast.LENGTH_LONG).show()
}
```

18. Una vez creados los listener para ambos controles. Ejecuta la aplicación. Y comprueba su funcionamiento.

Tercera Parte: Añadir un spinner.

En esta tercera parte vamos a añadir un **spinner** (que incluya una lista por ejemplo, Provincias, Islas, Series, películas, juegos, etc), un **TextView** para mostrar el ítem del spinner seleccionado

19. Añade los diferentes componentes **Spinner** y (**TextView**), en la siguiente imagen puedes ver un ejemplo..



20. Ahora accedemos al spinner y el TextView añadidos usando **binding.LaIdDelComponente**

```
// Accedemos al spinner y TextView que acabamos de añadir
val espiner = binding.espin
val tvespin = binding.tvespin
```

21. **Spinner**: es un componente permite seleccionar un valor de entre un conjunto, lo que conocemos como ComboBox. Este grupo de valores puede almacenarse en un array o bien, en un cursor si lo cargamos desde una base de datos.

22. También tenemos la posibilidad de definir el array en el fichero **strings.xml**. Esta opción no la haremos puedes ver un ejemplo de como se haría en la siguiente imagen.

```
<resources>
    <string name="app_name">Viewbinding24</string>
    <string-array name="provincias">
        <item>A Coruña</item>
        <item>Lugo</item>
        <item>Orense</item>
        <item>Pontevedra</item>
    </string-array>
</resources>
```

23. Ahora puedes ver como se crearía el adaptador para vincular nuestros datos con el Spinner. Para ello utilizando el **createFromResource()** para hacer referencia al array XML que creamos en strings.xml y luego asignamos el adaptador al spinner.

```
val pro = ArrayAdapter.createFromResource(context: this, R.array.provincias, com.google.android.material.R.layout.support_simple_spinner_dropdown_item)
espin.adapter = pro
```

24. La opción que vamos a realizar es crear el array en Kotlin es la que realizaremos el código sería el siguiente:

```
// Creamos el array con las Islas Baleares
val baleares = arrayOf("Mallorca", "Menorca", "Ibiza", "Formentera", "Cabrera")
// Creamos el adaptador que está en nuestra Activity para un spinner y asignamos el array creado
val adapta = ArrayAdapter(context = this, resource = androidx.appcompat.R.layout.support_simple_spinner_dropdown_item, objects = baleares)
// Vinculamos el adaptador creado al del spinner
espin.adapter = adapta
```

25. Ahora vamos a ver los eventos lanzados por el Spinner, el más utilizado será el generado al seleccionarse una opción de la lista desplegable, **onItemSelected**. Para capturar este evento se procederá como lo hemos visto para otros controles anteriormente, asignándole su controlador mediante la propiedad **onItemSelectedListener**. A diferencia de los controles anteriores, para este evento definimos dos métodos, el primero de ellos (**onItemSelected**) que será llamado cada vez que se seleccione una opción en la lista desplegable, y el segundo (**onNothingSelected**) que se llamará cuando no haya ninguna opción seleccionada

(esto puede ocurrir por ejemplo si el adaptador no tiene datos). Además, vemos cómo para recuperar el dato seleccionado utilizamos el método **getItemAtPosition()** del parámetro **AdapterView (p0 en el ejemplo)** que recibimos en el evento. En este ejemplo mostramos el valor seleccionado en el TextView para el mensaje.

```
espiner.onItemSelectedListener = object : AdapterView.OnItemSelectedListener {  
    override fun onItemSelected(  
        p0: AdapterView<*>?,  
        p1: View?,  
        p2: Int,  
        p3: Long  
    ) {  
        tvespin.text = "Seleccionaste ${p0!!.getItemAtPosition( position = p2)}"  
    }  
  
    override fun onNothingSelected(p0: AdapterView<*>?) {  
        tvespin.text = "Nada seleccionado"  
    }  
}
```

26. Ejecuta el programa y observa su funcionamiento..

27. A continuación, puedes ver la aplicación de ejemplo en funcionamiento en el siguiente video:

<https://drive.google.com/file/d/1Niqb5dZUzkjwQCxukzTtLuFDoODkSTLX/view?usp=sharing>